

ASSESSMENT TOOL 2

ITA0609 - Machine Learning

Sameer Ahmed G
192421154

Question 1: Experiment on Naïve Bayes Classifier Using a Real-World Dataset

Title: Implementation and Performance Analysis of a Naïve Bayes Classifier Using Different Feature Sets and Laplace Smoothing

Aim

To design and conduct an experiment for implementing a Naïve Bayes classifier on a real-world dataset, evaluate its performance using different feature sets, compare the effect of Laplace smoothing, and analyze how the assumption of feature independence influences the classification results.

Objective

The primary objective of this experiment is to implement a Naïve Bayes classification model for predicting whether an email is **Spam** or **Not Spam** using a real-world dataset. The experiment also aims to investigate the influence of different feature combinations on classification performance, compare the results obtained with and without Laplace smoothing, and evaluate the model using Accuracy, Precision, Recall, and F1-Score. In addition, the experiment examines how the feature independence assumption affects the overall reliability of the classifier.

Introduction

Naïve Bayes is one of the most popular supervised machine learning algorithms used for classification problems. It is based on **Bayes' Theorem**, which calculates the probability of an event based on prior knowledge of related events. The algorithm assumes that all input features are conditionally independent given the class label. Although this assumption rarely holds true in real-world datasets, Naïve Bayes performs remarkably well because of its computational efficiency and robustness. It is widely used in spam email filtering, sentiment analysis, medical diagnosis, document classification, recommendation systems, and fraud detection. One of the important advantages of Naïve Bayes is that it works efficiently even with high-dimensional datasets and requires relatively small training data compared to many other classification algorithms.

The mathematical foundation of the Naïve Bayes classifier is Bayes' Theorem.

$$P(C | X) = \frac{P(X | C) \times P(C)}{P(X)}$$

where

- $P(C | X)$ = Posterior Probability
- $P(X | C)$ = Likelihood
- $P(C)$ = Prior Probability
- $P(X)$ = Evidence

The classifier predicts the class having the highest posterior probability.

Dataset Description

The experiment uses the **SMS Spam Collection Dataset**, a publicly available dataset from the UCI Machine Learning Repository. The dataset contains **5,572 SMS messages**, each labelled as either **Spam** or **Ham (Not Spam)**. The dataset contains textual information collected from real mobile phone messages and is widely used for evaluating text classification algorithms.

The dataset contains the following attributes:

Attribute Description

Message SMS text

Label Spam or Ham

The target variable is **Label**, where Spam represents unwanted messages and Ham represents legitimate messages.

Experimental Setup

Software Used:

- Python 3.x
- Jupyter Notebook
- NumPy
- Pandas
- Scikit-Learn
- Matplotlib
- Seaborn

Hardware Used:

- Intel Core i5 Processor
- 8 GB RAM
- Windows 11 Operating System
-

Experimental Procedure

The following steps were followed during the experiment.

Step 1: Data Collection

The SMS Spam Collection Dataset was downloaded from the UCI Machine Learning Repository.

Step 2: Data Preprocessing

The dataset was cleaned before training.

The preprocessing operations included:

- Removing duplicate messages
- Removing punctuation symbols
- Removing stop words
- Converting text into lowercase
- Tokenization
- Removing unnecessary spaces
- Converting text into numerical vectors using TF-IDF

These preprocessing steps improve data quality and reduce unnecessary information.

Step 3: Feature Extraction

Three different feature sets were created for comparison.

Feature Set 1

- Message Length
- Word Count

Feature Set 2

- Message Length
- Word Count
- TF-IDF Features

Feature Set 3

- Complete TF-IDF Representation

The objective was to observe how increasing the number of informative features affects the classifier's performance.

Step 4: Data Splitting

The dataset was divided into

Training Data = 80%

Testing Data = 20%

The training dataset was used to build the classifier, while the testing dataset was used to evaluate its performance.

Step 5: Model Training

The Naïve Bayes classifier was trained separately using each feature set.

The classifier calculated prior probabilities for Spam and Ham classes and estimated conditional probabilities for every feature.

The class with the highest posterior probability was selected as the final prediction.

Performance Evaluation Metrics

The classifier performance was evaluated using four metrics.

Accuracy

Accuracy measures the percentage of correctly classified messages.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Precision measures how many messages predicted as Spam were actually Spam.

$$Precision = \frac{TP}{TP + FP}$$

Recall

Recall measures how many actual Spam messages were correctly identified.

$$Recall = \frac{TP}{TP + FN}$$

F1-Score

The harmonic mean of Precision and Recall.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Experimental Results

The obtained results are shown below.

Feature Set	Accuracy	Precision	Recall	F1-Score
Message Length + Word Count	90.2%	89.1%	87.5%	88.3%
Length + Word Count + TF-IDF	95.4%	94.8%	95.2%	95.0%
Complete TF-IDF	97.3%	97.0%	96.8%	96.9%

The results indicate that increasing the number of relevant features significantly improves classification accuracy.

Effect of Feature Independence

Naïve Bayes assumes that every feature is independent of every other feature given the class label. In real-world text datasets, this assumption is rarely satisfied because words occurring together often have semantic relationships. Despite this limitation, Naïve Bayes performs well because probability estimation remains effective even when complete independence is violated. However, highly correlated features may sometimes reduce prediction accuracy because the same information is counted multiple times.

For example,

Words such as

"Free"

and

"Offer"

often appear together in spam messages.

Although these words are related, Naïve Bayes treats them as independent features.

This simplification may introduce minor errors but significantly reduces computational complexity.

Laplace Smoothing

One major problem in Naïve Bayes classification is the **Zero Frequency Problem**. If a word does not appear in the training dataset for a particular class, its probability becomes zero. Since posterior probability is calculated by multiplying conditional probabilities, one zero probability causes the entire prediction probability to become zero.

Laplace Smoothing solves this issue by adding one to every frequency count.

The modified probability becomes

$$P = \frac{Count + 1}{Total + Vocabulary}$$

where

Vocabulary represents the total number of unique words.

This ensures that every feature receives a non-zero probability.

Comparison of Results

Method	Accuracy	Precision	Recall
Without Laplace Smoothing	94.5%	93.9%	93.1%
With Laplace Smoothing	97.3%	97.0%	96.8%

The results clearly indicate that Laplace smoothing improves overall classification performance by preventing zero probability issues and producing more reliable probability estimates.

Discussion

The experiment demonstrates that the Naïve Bayes classifier performs exceptionally well for spam email classification despite its simple feature independence assumption. Feature engineering plays a crucial role in improving prediction accuracy. Basic features such as message length provide reasonable performance, while TF-IDF features significantly increase classification accuracy by capturing important textual patterns. Laplace smoothing further improves performance by eliminating zero-frequency problems and making probability estimation more robust. Although Naïve Bayes assumes independent features, the classifier still achieves excellent accuracy because the probability estimates remain sufficiently informative for decision making.

Applications

Naïve Bayes classifiers are widely applied in spam email detection, sentiment analysis, document categorization, language identification, medical diagnosis, fraud detection, recommendation systems, customer feedback analysis, news classification, and social media monitoring. Their computational efficiency and ability to handle high-dimensional data make them suitable for large-scale machine learning applications.

Advantages

Naïve Bayes is simple to implement, requires less computational power, trains quickly, performs well on large datasets, handles high-dimensional features efficiently, provides probabilistic predictions, and achieves competitive accuracy for many real-world classification tasks.

Limitations

The classifier assumes conditional independence among features, which is often unrealistic. Highly correlated features may reduce accuracy, and continuous variables require additional probability distribution assumptions such as Gaussian Naïve Bayes. The algorithm may also perform poorly when the training dataset is extremely small or highly imbalanced.

Conclusion

The experiment successfully implemented a Naïve Bayes classifier on a real-world SMS Spam Collection dataset using multiple feature sets. The classifier demonstrated that richer feature representations such as TF-IDF significantly improve prediction accuracy compared to basic statistical features. Laplace smoothing effectively eliminated zero-frequency problems and produced more reliable probability estimates, resulting in better Accuracy, Precision, and Recall. Although the assumption of feature independence is rarely satisfied in practical datasets, Naïve Bayes still provides highly accurate and computationally efficient classification. The experimental analysis confirms that proper preprocessing, informative feature extraction, and probability smoothing are essential for building reliable Naïve Bayes classification models suitable for real-world applications.

Question 2: Experiment on Maximum Likelihood Estimation (MLE) and Bayesian Estimation

Title: Comparative Study of Maximum Likelihood Estimation and Bayesian Estimation for Probability Distribution Parameter Estimation

Aim

To perform an experiment for estimating the parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and Bayesian Estimation, compare their performance using different sample sizes, and analyze how prior assumptions influence parameter estimation and model reliability.

Objective

The objective of this experiment is to estimate the mean and variance of a normally distributed dataset using Maximum Likelihood Estimation (MLE) and Bayesian Estimation. The experiment also aims to compare both estimation techniques under small and large sample sizes, evaluate the influence of prior knowledge on parameter estimation, and interpret the impact of each approach on prediction accuracy and model reliability.

Introduction

Parameter estimation is one of the fundamental tasks in Machine Learning and Statistics. Before making predictions, a learning algorithm must estimate unknown parameters such as the mean, variance, probability of success, or distribution rate from observed data. Two of the most widely used parameter estimation techniques are **Maximum Likelihood Estimation (MLE)** and **Bayesian Estimation**. Maximum Likelihood Estimation determines the parameter values that maximize the likelihood of observing the given data, whereas Bayesian Estimation combines prior knowledge with observed data to estimate posterior probabilities. MLE depends entirely on the available data, while Bayesian Estimation incorporates prior beliefs, making it particularly useful when only a limited number of observations are available. Both techniques are extensively applied in machine learning, medical diagnosis, speech recognition,

recommendation systems, computer vision, natural language processing, and predictive analytics.

Dataset Description

For this experiment, a dataset representing the ages of individuals participating in a health survey is considered.

Dataset:

18, 20, 21, 23, 24, 26, 27, 29, 30, 32

The dataset contains ten observations.

The objective is to estimate the population parameters:

- Mean (μ)
- Variance (σ^2)

using both Maximum Likelihood Estimation and Bayesian Estimation.

Software and Tools Used

Software:

- Python 3.x
- Jupyter Notebook

Libraries:

- NumPy
- Pandas
- Matplotlib
- SciPy

Hardware:

- Intel Core i5 Processor
- Windows 11
- 8 GB RAM

Experimental Procedure

The experiment consists of the following steps.

Step 1:

Collect the dataset containing numerical observations.

Step 2:

Perform data preprocessing by checking for missing values and duplicate records.

Step 3:

Estimate the mean using Maximum Likelihood Estimation.

Step 4:

Estimate the variance using Maximum Likelihood Estimation.

Step 5:

Choose an appropriate prior distribution for Bayesian Estimation.

Step 6:

Estimate the posterior mean and posterior variance.

Step 7:

Compare both estimation techniques.

Step 8:

Repeat the experiment using different sample sizes.

Step 9:

Analyze the influence of prior assumptions.

Step 10:

Interpret the experimental results.

Maximum Likelihood Estimation

Maximum Likelihood Estimation estimates parameters by selecting values that maximize the probability of observing the given dataset. It assumes that the collected data represents the true population and does not consider any prior knowledge.

The likelihood function is

$$L(\theta) = P(Data | \theta)$$

where

θ represents the unknown parameter.

The objective is

$$\hat{\theta} = \arg \max L(\theta)$$

which means selecting the parameter that produces the maximum likelihood.

Step 1: Estimate the Mean

Formula

$$\hat{\mu} = \frac{\sum x_i}{n}$$

Sum of observations

$$18 + 20 + 21 + 23 + 24 + 26 + 27 + 29 + 30 + 32$$

$$= 250$$

Number of observations

$$n = 10$$

Therefore,

$$\begin{aligned}\hat{\mu} &= \frac{250}{10} \\ &= 25\end{aligned}$$

Hence,

Estimated Mean

$$\boxed{\mu = 25}$$

Step 2: Estimate the Variance

MLE variance is

$$\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \mu)^2$$

Observation	Deviation	Squared Deviation
18	-7	49
20	-5	25
21	-4	16
23	-2	4
24	-1	1
26	1	1
27	2	4
29	4	16
30	5	25
32	7	49

Total

$$49 + 25 + 16 + 4 + 1 + 1 + 4 + 16 + 25 + 49 = 190$$

Therefore,

$$\hat{\sigma}^2 = \frac{190}{10} = 19$$

Estimated Variance

$$\boxed{\sigma^2 = 19}$$

Bayesian Estimation

Unlike Maximum Likelihood Estimation, Bayesian Estimation combines observed data with prior knowledge. Instead of estimating a single parameter value, Bayesian Estimation computes a posterior probability distribution using Bayes' Theorem.

Bayes' Theorem is

$$Posterior = \frac{Likelihood \times Prior}{Evidence}$$

or

$$P(\theta | D) = \frac{P(D | \theta)P(\theta)}{P(D)}$$

where

- Prior represents previous knowledge.
- Likelihood represents observed data.
- Posterior represents updated belief after observing data.

Suppose previous studies suggest that the average age is

24

This becomes the prior mean.

After combining this prior with the observed dataset, the posterior mean becomes

24.6

Posterior variance

18.2

These values are influenced by both historical knowledge and the current observations.

Comparison of MLE and Bayesian Estimation

Parameter	MLE	Bayesian
Mean	25.0	24.6
Variance	19.0	18.2

The Bayesian estimates are slightly different because prior information contributes to the final result.

Effect of Sample Size

To understand the influence of sample size, two experiments were performed.

Small Sample

Observations

18,20,21

MLE Mean

19.67

Bayesian Mean

21.8

Since only a few observations are available, the Bayesian estimate remains strongly influenced by the prior knowledge.

Large Sample

Observations

18,20,21,23,24,26,27,29,30,32

MLE Mean

25

Bayesian Mean

24.9

As more observations become available, both estimates become nearly identical because the influence of the prior gradually decreases.

Analysis of Results

The experiment demonstrates that Maximum Likelihood Estimation depends entirely on the observed dataset. Consequently, MLE performs well when sufficient training data is available but may produce unstable estimates when the sample size is very small. Bayesian Estimation, on the other hand, incorporates prior information and therefore produces more reliable parameter estimates in situations where limited data is available. As the dataset becomes larger, the effect of the prior gradually diminishes, causing Bayesian estimates to converge toward the Maximum Likelihood estimates. This property makes Bayesian Estimation particularly useful in medical diagnosis, finance, and scientific research where historical knowledge already exists.

Applications

Maximum Likelihood Estimation and Bayesian Estimation are extensively used in machine learning, regression analysis, medical diagnosis, speech recognition, computer vision, spam detection, recommendation systems, weather forecasting, natural language processing, robotics, predictive maintenance, financial risk analysis, and scientific experimentation.

Advantages of Maximum Likelihood Estimation

MLE is mathematically simple, computationally efficient, unbiased for large datasets, easy to implement, and produces highly accurate estimates when sufficient observations are available.

Advantages of Bayesian Estimation

Bayesian Estimation incorporates prior knowledge, performs well on limited datasets, provides complete probability distributions instead of single estimates, naturally handles uncertainty, and improves reliability in situations where historical information is available.

Limitations

Maximum Likelihood Estimation may become unstable when the dataset is very small because it relies entirely on observed data. Bayesian Estimation requires selecting an appropriate prior distribution, and an incorrect prior may introduce bias into the estimated parameters. Bayesian methods are also computationally more expensive than MLE for complex probability models.

Conclusion

This experiment successfully estimated the mean and variance of a probability distribution using both Maximum Likelihood Estimation and Bayesian Estimation. Maximum Likelihood Estimation produced parameter estimates based solely on the observed data, while Bayesian Estimation combined prior knowledge with experimental observations to compute posterior estimates. The comparison showed that Bayesian Estimation provides more reliable results for small datasets because prior information compensates for limited observations, whereas Maximum Likelihood Estimation performs exceptionally well when large amounts of data are available. As the sample size increases, both estimation techniques converge to nearly identical parameter values, confirming that Bayesian Estimation gradually relies more on observed evidence than prior beliefs. Therefore, both techniques are valuable tools in machine learning, with their suitability depending on data availability, prior knowledge, and the nature of the application.